Structural Design Patterns Report — Bank Management System

1.1 Table of Contents

1. Bridge Pattern — Use 1: Interest Posting
2. Bridge Pattern — Use 2: Loan Application Processing
3. Adapter Pattern — Use 1: Database SQL-Dialect Adapter
4. Adapter Pattern — Use 2: Payment Gateway
5. Decorator Pattern — Use 1: Account Info Service
6. Decorator Pattern — Use 2: Transaction Processor
7. Flyweight Pattern — Loan Product Catalog

1.2 1. Bridge Pattern — Use 1: Interest Posting

1.2.1 Problem Identified

The system posts monthly interest to different account types (Savings, Money Market), each with a distinct interest rate and calculation formula. Without the Bridge pattern, supporting *N* account types × *M* interest algorithms would require N×M subclasses, creating a **combinatorial explosion** of tightly coupled classes. Any change to the posting workflow or a new interest formula would ripple across many classes.

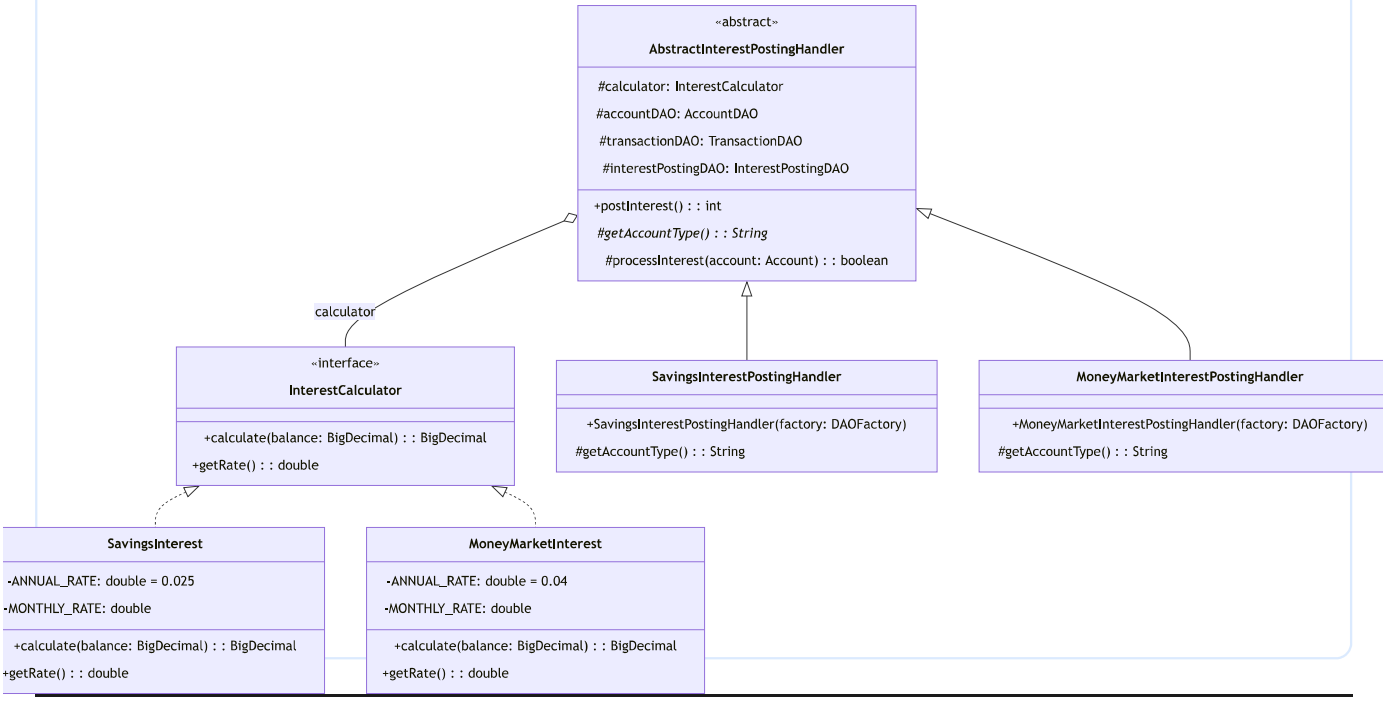
1.2.2 Justification

The **Bridge** pattern decouples the *abstraction* (the interest-posting workflow) from its *implementation* (the interest-calculation algorithm). This allows: - **Independent variation**: New account types can be added by subclassing AbstractInterestPostingHandler, and new interest formulas by implementing InterestCalculator, without modifying existing code. - **Open/Closed Principle**: Both hierarchies are open for extension, closed for modification. - **Testability**: The InterestCalculator can be mocked independently of the posting workflow.

1.2.3 Participating Classes

Role	Class	Package
Abstraction	AbstractInterestPostingHandler	`domain.controller`
Refined Abstraction	SavingsInterestPostingHandler	`domain.controller`
Refined Abstraction	MoneyMarketInterestPostingHandler	`domain.controller`
Implementor	InterestCalculator	`domain.controller`
Concrete Implementor	SavingsInterest	`domain.controller`
Concrete Implementor	MoneyMarketInterest	`domain.controller`

1.2.4 UML Class Diagram



1.3 2. Bridge Pattern — Use 2: Loan Application Processing

1.3.1 Problem Identified

Loan applications can be of several types (Personal, Home, Auto) and each has its own interest-rate calculation strategy. The loan-processing workflow (validation, persistence, status management) is common across types, but the rate computation differs. Combining both concerns in a single class hierarchy would again create a combinatorial explosion and would violate the **Single Responsibility Principle**.

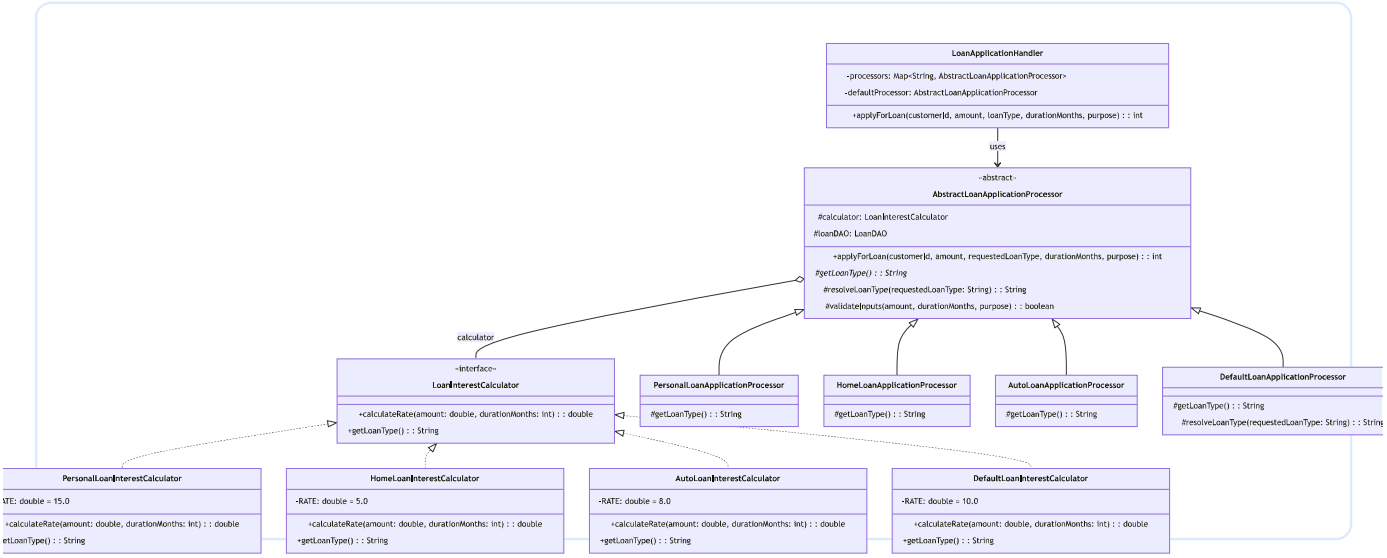
1.3.2 Justification

The **Bridge** pattern decouples the *loan application workflow* (abstraction) from the *interest-rate calculation strategy* (implementation): - **Abstraction hierarchy**: AbstractLoanApplicationProcessor defines the application workflow; refined abstractions like PersonalLoanApplicationProcessor fix the loan type. - **Implementor hierarchy**: LoanInterestCalculator defines the rate-computation interface; concrete implementations provide type-specific rates. - New loan types only need a new refined abstraction + a new calculator without modifying existing code.

1.3.3 Participating Classes

Role	Class	Package
Abstraction	AbstractLoanApplicationProcessor	`domain.controller`
Refined Abstraction	PersonalLoanApplicationProcessor	`domain.controller`
Refined Abstraction	HomeLoanApplicationProcessor	`domain.controller`
Refined Abstraction	AutoLoanApplicationProcessor	`domain.controller`
Refined Abstraction	DefaultLoanApplicationProcessor	`domain.controller`
Implementor	LoanInterestCalculator	`domain.controller`
Concrete Implementor	PersonalLoanInterestCalculator	`domain.controller`
Concrete Implementor	HomeLoanInterestCalculator	`domain.controller`
Concrete Implementor	AutoLoanInterestCalculator	`domain.controller`
Concrete Implementor	DefaultLoanInterestCalculator	`domain.controller`
Client	LoanApplicationHandler	`domain.controller`

1.3.4 UML Class Diagram



1.4 3. Adapter Pattern — Use 1: Database SQL-Dialect Adapter

1.4.1 Problem Identified

The BMS persistence layer must support multiple relational database engines (SQL Server, MySQL, Oracle, PostgreSQL), each with incompatible SQL dialects for pagination, auto-generated keys, timestamps, date formatting, and feature support. The existing DAO classes are written against a **generic interface**, but each database engine provides a **vendor-specific API** that is incompatible.

1.4.2 Justification

The **Adapter** pattern (Object Adapter variant) is used to present a uniform DatabaseAdapter interface that adapts the vendor-specific SQL differences to a common contract: - **Target**: The DatabaseAdapter interface that the DAO layer and DataSourceFactory expect. - **Adaptees**: The vendor-specific SQL quirks (e.g., MySQL's `LIMIT ?,?`, Oracle's `OFFSET … ROWS FETCH NEXT`, PostgreSQL's `RETURNING id`). - **Adapters**: MySQLAdapter, OracleAdapter, PostgreSQLAdapter, SqlServerAdapter — each encapsulates the dialect-specific logic behind the common interface.

This allows the DAO layer to remain database-agnostic and swap backends through configuration alone.

1.4.3 Participating Classes

Role	Class	Package
Target	DatabaseAdapter	`persistence.adapter`
Adapter	MySQLAdapter	`persistence.adapter`
Adapter	OracleAdapter	`persistence.adapter`
Adapter	PostgreSQLAdapter	`persistence.adapter`
Adapter	SqlServerAdapter	`persistence.adapter`
Factory	DatabaseAdapterFactory	`persistence.adapter`
Client	DataSourceFactory	`persistence`

1.4.4 UML Class Diagram

1.5 4. Adapter Pattern — Use 2: Payment Gateway

1.5.1 Problem Identified

The BMS needs to process payments through external third-party providers (Stripe, PayPal, Square) when executing fund transfers. Each provider exposes a **proprietary, incompatible API**: Stripe uses API keys and amounts in cents, PayPal uses OAuth client credentials and order objects, and Square uses access tokens with location IDs and its own currency format. The domain layer (TransferHandler) should not couple to any vendor SDK.

1.5.2 Justification

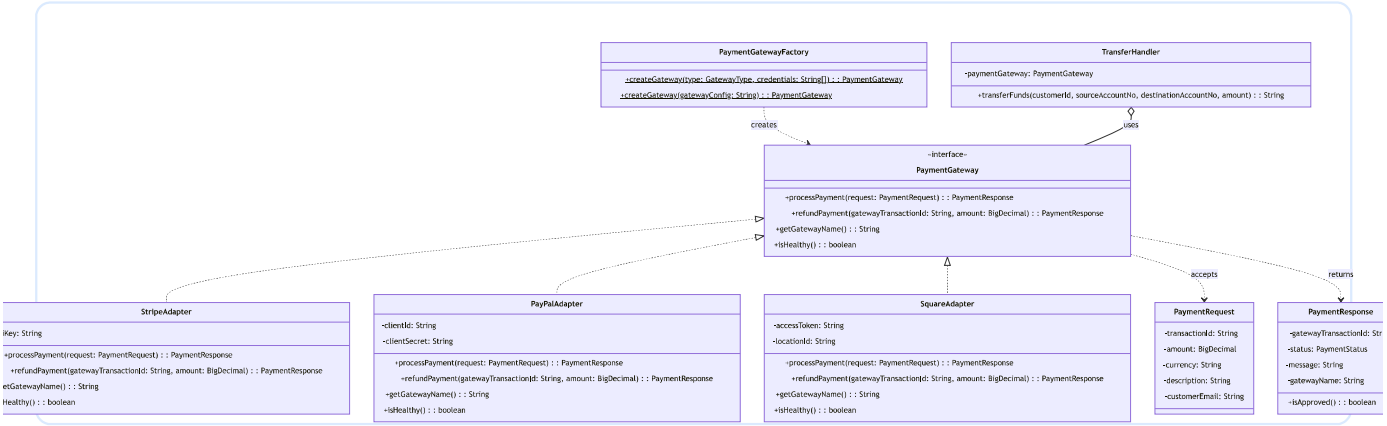
The **Adapter** pattern (Object Adapter variant) introduces a uniform PaymentGateway interface that the domain layer programs against: - **Target**: PaymentGateway — the contract TransferHandler expects. - **Adaptees**: The vendor-specific APIs (Stripe Charge API, PayPal Orders API, Square Payments API). - **Adapters**: `StripeAdapter`, `PayPalAdapter`, `SquareAdapter` — each encapsulates the vendor-specific authentication, request conversion, and response translation behind the common PaymentGateway interface.

This allows the system to switch payment providers through configuration alone, without modifying domain logic.

1.5.3 Participating Classes

Role	Class	Package
Target	PaymentGateway	payment
Adapter	StripeAdapter	payment
Adapter	PayPalAdapter	payment
Adapter	SquareAdapter	payment
Factory	PaymentGatewayFactory	payment
Request DTO	PaymentRequest	payment
Response DTO	PaymentResponse	payment
Client	TransferHandler	`domain.controller`

1.5.4 UML Class Diagram



1.6 5. Decorator Pattern — Use 1: Account Info Service

1.6.1 Problem Identified

When displaying an account balance, different presentation concerns must be layered onto the basic account data: **currency formatting**, **overdraft warnings**, and **reward-point calculations**. Using subclassing to combine these features would lead to an explosion of classes (e.g., `CurrencyFormattedService`, `CurrencyFormattedWithWarningService`, `CurrencyFormattedWithRewardsService`, etc.). Furthermore, different screens may require different combinations of these enhancements.

1.6.2 Justification

The **Decorator** pattern allows features to be stacked dynamically at runtime via composition: - Each decorator wraps an AccountInfoService and adds one concern. - Decorators are composable — AccountBalanceController chains `CurrencyFormatDecorator → OverdraftWarningDecorator → RewardPointsDecorator`. - Adding a new enhancement (e.g., loyalty tier labeling) requires only a new decorator class, no modification to existing code.

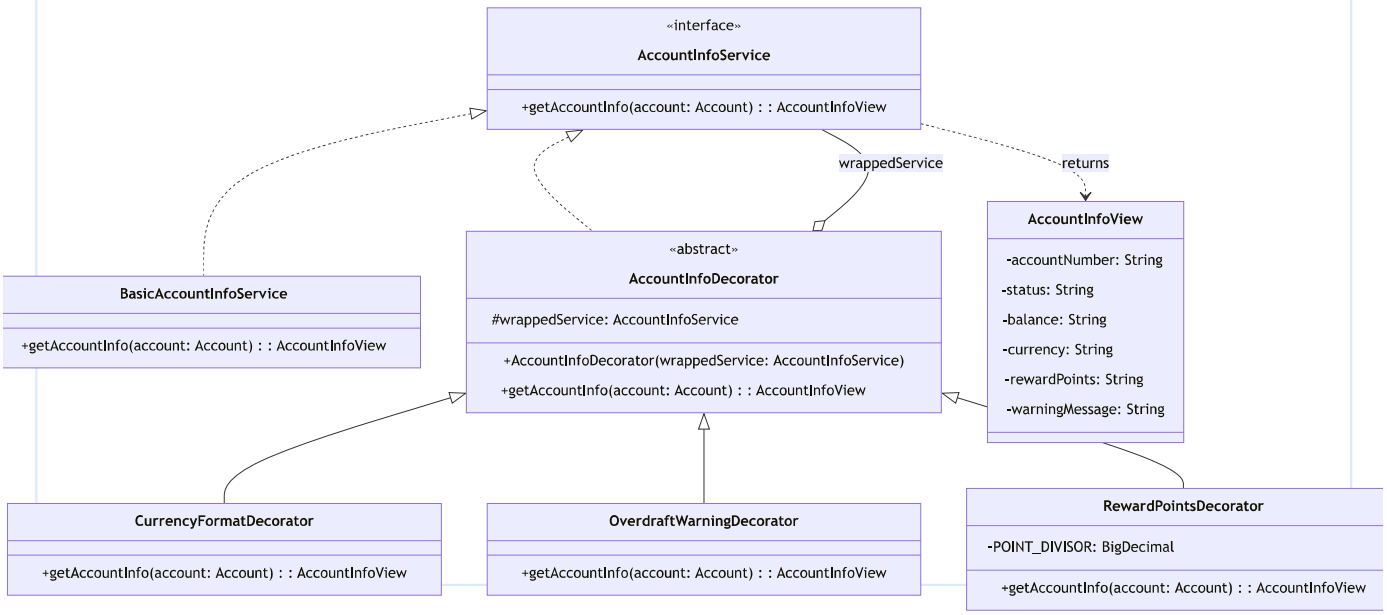
Usage in AccountBalanceController:

```
this.accountInfoService = new RewardPointsDecorator(
    new OverdraftWarningDecorator(
        new CurrencyFormatDecorator(
            new BasicAccountInfoService())));
```

1.6.3 Participating Classes

Role	Class	Package
Component	AccountInfoService	`domain.decorator.accountinfo`
Concrete Component	BasicAccountInfoService	`domain.decorator.accountinfo`
Base Decorator	AccountInfoDecorator	`domain.decorator.accountinfo`
Concrete Decorator	CurrencyFormatDecorator	`domain.decorator.accountinfo`
Concrete Decorator	OverdraftWarningDecorator	`domain.decorator.accountinfo`
Concrete Decorator	RewardPointsDecorator	`domain.decorator.accountinfo`
Data Transfer Object	AccountInfoView	`domain.decorator.accountinfo`

1.6.4 UML Class Diagram



1.7 6. Decorator Pattern — Use 2: Transaction Processor

1.7.1 Problem Identified

Transaction processing needs cross-cutting concerns applied around the core operation: **audit logging**, **fee calculation**, and **notification dispatch**. These concerns are orthogonal — some transactions need fees but not audit, others need all three. Hard-coding every combination into the handler classes would produce rigid, duplicated code.

1.7.2 Justification

The **Decorator** pattern wraps the core TransactionProcessor with optional cross-cutting behaviors: - BasicTransactionProcessor executes the raw operation. - AuditDecorator adds audit logging before and after. - FeeDecorator applies transaction fees before processing. - NotificationDecorator sends a notification after processing. - Decorators can be freely composed per use case.

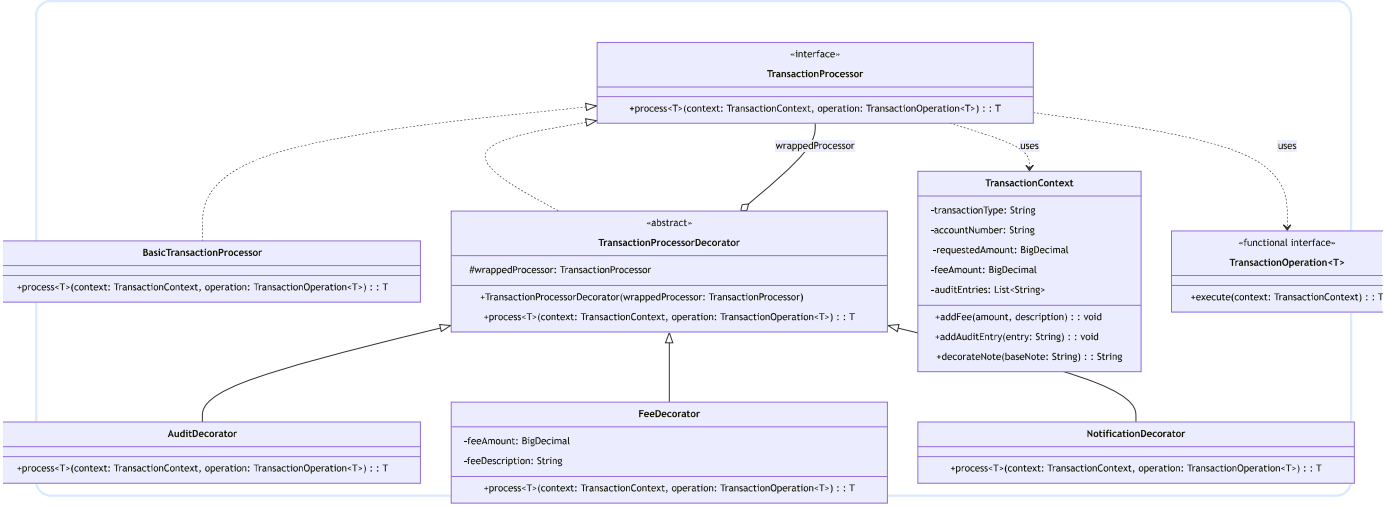
Usage in DepositHandler:

```
this.transactionProcessor = new NotificationDecorator(
    new AuditDecorator(
        new BasicTransactionProcessor()));
```

1.7.3 Participating Classes

Role	Class	Package
Component	TransactionProcessor	`domain.decorator.transaction`
Concrete Component	BasicTransactionProcessor	`domain.decorator.transaction`
Base Decorator	TransactionProcessorDecorator	`domain.decorator.transaction`
Concrete Decorator	AuditDecorator	`domain.decorator.transaction`
Concrete Decorator	FeeDecorator	`domain.decorator.transaction`
Concrete Decorator	NotificationDecorator	`domain.decorator.transaction`
Context	TransactionContext	`domain.decorator.transaction`
Operation	TransactionOperation	`domain.decorator.transaction`

1.7.4 UML Class Diagram



1.8 7. Flyweight Pattern — Loan Product Catalog

1.8.1 Problem Identified

The Loan Catalog and Comparison feature allows customers to compare loan products side-by-side for arbitrary amounts and durations. Each comparison scenario includes significant **shared, immutable product metadata** (display name, description, rate policy label, eligibility notes, interest calculator). Without sharing, every comparison scenario would duplicate this metadata, wasting memory — especially problematic when many customers run comparisons concurrently.

1.8.2 Justification

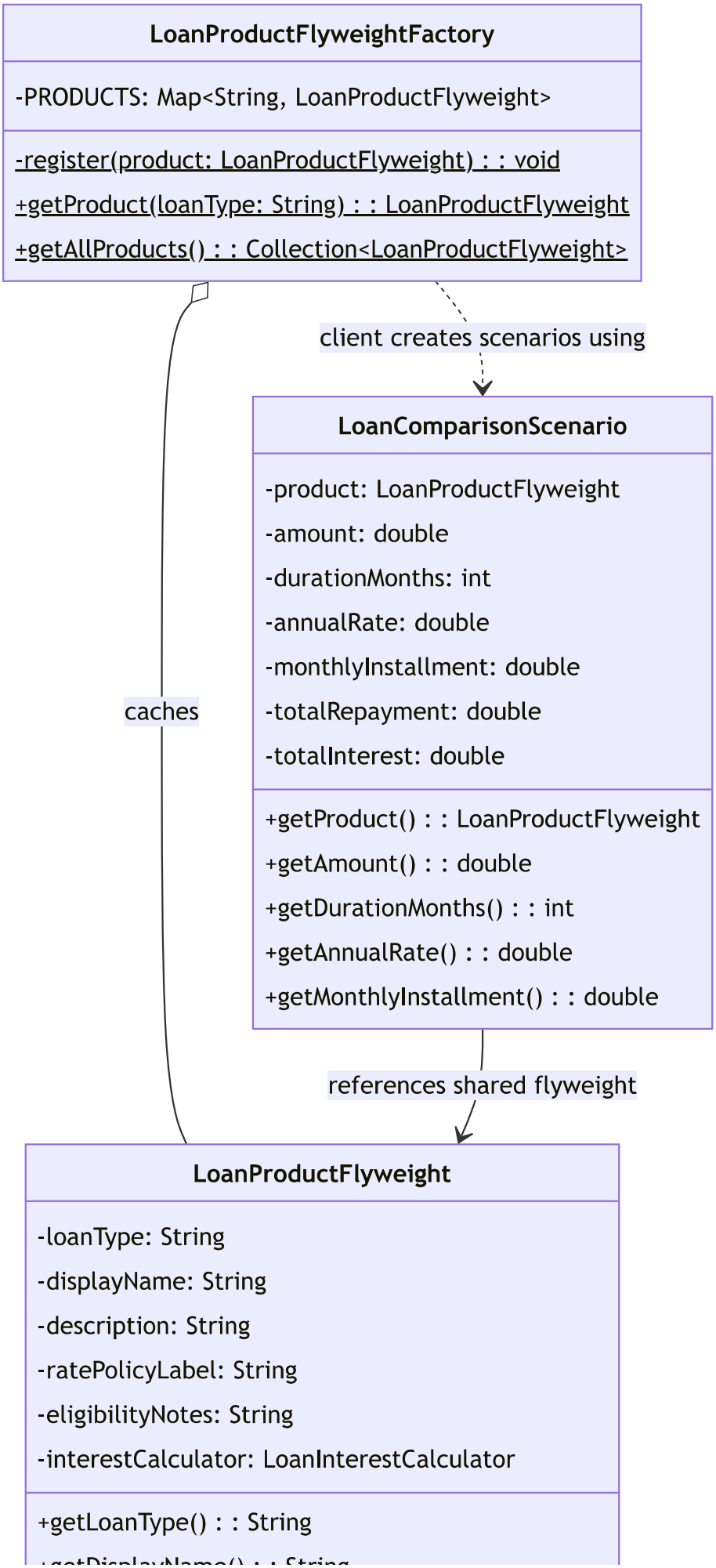
The **Flyweight** pattern separates **intrinsic state** (shared product metadata — immutable, identical across all scenarios) from **extrinsic state** (per-scenario customer inputs — amount, duration, calculated installments): - LoanProductFlyweight is the flyweight object holding intrinsic state. - LoanProductFlyweightFactory is the flyweight factory that caches and reuses flyweight instances (keyed by loan type). - LoanComparisonScenario holds the extrinsic state (amount, duration, calculated results) and references a shared flyweight.

Usage in LoanCatalogController: for each comparison, scenarios are built using shared LoanProductFlyweight instances from the factory, avoiding duplicating product metadata per scenario.

1.8.3 Participating Classes

Role	Class	Package
Flyweight	LoanProductFlyweight	`domain.flyweight.loan`
Flyweight Factory	LoanProductFlyweightFactory	`domain.flyweight.loan`
Extrinsic Context	LoanComparisonScenario	`domain.flyweight.loan`
Client	LoanCatalogComparisonView	`presentation`

1.8.4 UML Class Diagram


LoanProductFlyweightFactory
-PRODUCTS: Map<String, LoanProductFlyweight>
-register(product: LoanProductFlyweight) : : void
+getProduct(loanType: String) : : LoanProductFlyweight
+getAllProducts() : : Collection<LoanProductFlyweight>
client creates scenarios using
caches
LoanComparisonScenario
-product: LoanProductFlyweight
-amount: double
-durationMonths: int
-annualRate: double
-monthlyInstallment: double
-totalRepayment: double
-totalInterest: double
+getProduct() : : LoanProductFlyweight
+getAmount() : : double
+getDurationMonths() : : int
+getAnnualRate() : : double
+getMonthlyInstallment() : : double
references shared flyweight
LoanProductFlyweight
-loanType: String
-displayName: String
-description: String
-ratePolicyLabel: String
-eligibilityNotes: String
-interestCalculator: LoanInterestCalculator
+getLoanType() : : String

+getDisplayName() : : String
+getDescription() : : String
+getRatePolicyLabel() : : String
+getEligibilityNotes() : : String
+calculateRate(amount, durationMonths) : : double

1.9 Summary

#	Pattern	Use Case	Abstraction / Target	Implementation / Adaptee
1	Bridge	Interest Posting	AbstractInterestPostingHandler	InterestCalculator
2	Bridge	Loan Application	AbstractLoanApplicationProcessor	LoanInterestCalculator
3	Adapter	SQL Dialect	DatabaseAdapter	MySQL / Oracle / PostgreSQL / SqlServer
4	Adapter	Payment Gateway	PaymentGateway	Stripe / PayPal / Square adapters
5	Decorator	Account Info	AccountInfoService	Currency / Overdraft / RewardPoints decorators
6	Decorator	Transaction Processing	TransactionProcessor	Audit / Fee / Notification decorators
7	Flyweight	Loan Product Catalog	LoanProductFlyweight (shared)	LoanComparisonScenario (extrinsic)